第10章
类的成员

扫描全能王 创建

10.1.1 构造函数的定义

```
#include <iostream.h>
class Complex{
  double Real,Imag;
  public:
  void SetCom(double a,double b)
     { Real=a; Imag=b;}
};

void main()

{ Complex a,b;

   a.SetCom(1.0,2.0);

   b.SetCom(2.0,3.0);

}
```



```
{ Complex a(1.0,2.0);

  Complex b(2.0,3.0);

}
```

- 构造函数：

  在说明（产生）一个对象时自动被调用，所以一般用以初始化一个对象。

扫描全能王 创建

Complex(double a,double b):Real(a),Imag(b){}

```
using namespace std;
#include <iostream>
class Complex{
  double Real,Imag;
  public:
  Complex(double a,double b)
     { Real=a; Imag=b;}
  Complex()
     { Real=0; Imag=0;}
};

int main()

{ Complex a (1.0,2.0);

   Complex b, c();

}
```

❖构造函数的一些问题：

- 构造函数名字必须与类名相同；
- 不能指定函数返回值，也不能指定为void类型；
- 构造函数可带参数，可不带参数，也可指定参数的缺省值；
- 构造函数可重载；
- 在定义对象时，在对象名后加上一对括号后并不表示定义对象，也不表示要调用不带参数的构造函数，而表示一个不带参数的函数，它的返回值为指定类的对象；
- 静态对象首次创建后不再重新产生。

扫描全能王 创建

10.1.2 默认构造函数

- 默认构造函数：
  - 在定义类时，若没有定义类的构造函数，则编译器自动产生一个默认构造函数，该函数什么事也不做；
  - 在定义类时，若定义了类的构造函数，则编译器就不产生默认构造函数；
  - 在类中，若定义了没有参数的构造函数，或各参数均有缺省值的构造函数也称为默认构造函数，默认构造函数只能有一个；
  - 产生对象时，系统必定要调用构造函数，所以任一对象的构造函数必须唯一。

扫描全能王 创建

对于构造函数,下面哪一种说法是错误的?()

A 每个对象都是经过某个构造函数的执行才被创建出来的。

B 一个类可定义多个构造函数,它们形参不同。

C 构造函数中可对数据成员初始化。

D 构造函数中不能有return 语句。

扫描全能王 创建

对于缺省构造函数，下面哪一种说法是错误的？

A 类中没有显式定义任何构造函数时，编译器自动生成一个公有的缺省构造函数。

B 一个无参构造函数是缺省构造函数。

C 缺省构造函数一定是一个无参构造函数。

D 一个类中最多只能有一个缺省构造函数。

扫描全能王 创建

填空题 2分

假定AB为一个类，
若执行“AB a[3], b(3), *p;”语句时共调用该类构造函数的次数为([填空1]);
若执行“AB *q=new AB[5];”语句时共调用该类构造函数的次数为([填空2])。

扫描全能王 创建

10.1.3 委托构造函数

- 委托构造函数（C++11）：
  - 多个构造函数具有一些共同行为时，可利用委托构造函数避免重复编码；
  - 委托构造函数的语法形式如下：

    <类名>(形参表)：<类名>(实参表){函数体}
  - 先按实参表去调用构造函数，然后再执行自己的函数体。

扫描全能王 创建

单选题 1分

设有代码如下:

```
class Myclass{
public:
  Myclass()  { cout << "Myclass() "; }
  Myclass(int a) :Myclass()  { cout << "MyClass(int) "; }
  Myclass(double d) :Myclass()
        {cout << "MyClass(double)" << endl;}
};
```

执行Myclass(6.7);输出为_____。

A MyClass(double)

B MyClass() MyClass(double)

C MyClass()

D 编译错误

扫描全能王 创建

10.2 析构函数

- 析构函数：
  - 在一个对象消亡时被调用，因此常用来做善后工作，比如对象保持的动态空间的释放等。
- 类变量消亡：在变量退出其作用域时
  - 全局：程序结束
  - 局部：退出定义的分程序
  - 通过new得到的对象，用delete运算时。

扫描全能王 创建

10.2 析构函数

- 析构函数的一些问题：
  - 析构函数名字必须与类名相同，并在其前面加字符“~”；
  - 不能指定函数返回值，也不能指定为void类型；
  - 不可带参数；
  - 不允许重载；
  - 用delete运算释放一个对象会调用析构函数；
  - 在类变量消亡时，总要调用一个析构函数。

扫描全能王 创建

10.2 析构函数

```
using namespace std;
#include <cstring>
#include <iostream>
class Person{
   char *name;
public:
  Person(char *s)
   {  name=new char [strlen(s)+1];
      strcpy(name,s); }

void GetName(char *s)
    { strcpy(s,name);}

 ~Person(){delete name;}

int main()
{
     Person x ("张三"),y ("李四");
}
```

扫描全能王 创建

对于析构函数,下面哪一种说法是错误的?()

A 一个对象的生命周期始于构造函数执行,终于析构函数执行。

B 如果类中没有显式定义析构函数,编译器就自动生成一个公有的析构函数。

C 一个类中可重载定义多个析构函数。

D 析构函数中可用this 指针。

扫描全能王 创建

10.3.1 拷贝构造函数

- 格式：

  <类名>(const <类名>& <对象名>)：成员初始化表
  { 函数体 }
- 作用：一般用于完成类变量间的拷贝。
- 隐含拷贝构造函数：对没有定义构造函数的类，系统隐含有一个默认拷贝构造函数，功能是完成拷贝。

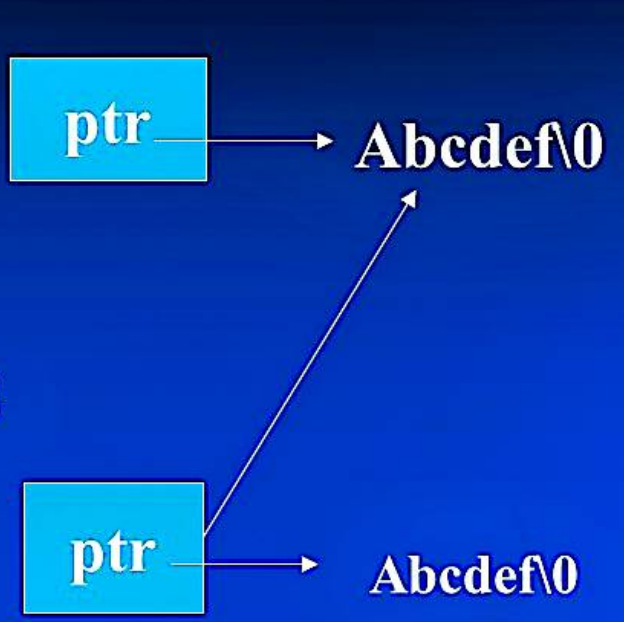
扫描全能王 创建

10.3.1 拷贝构造函数

- 调用拷贝构造函数的情况：
  - 用说明语句来创建一个对象时，用一个已有对象来初始化新建对象；
  - 调用某个函数时，以值传递一个命名对象。如果实参是匿名对象，则仅执行构造函数而不执行拷贝构造函数。
  - 函数返回一个对象时，返回匿名对象不执行拷贝构造函数。如果返回命名对象，是否执行拷贝构造函数与编译优化选项有关。

扫描全能王 创建

```
using namespace std;
#include <cstring>
#include <iostream>
class S{
        char *ptr;
public:
   S(char *p){
        if(!p) ptr=NULL;
        else {  ptr=new char[strlen(p)+1];
                strcpy(ptr,p);}
    }
   ~S(){ delete ptr;}
   S(S &);
};
int main()
{ S s1("abcdef");
  S s2=s1;     // S s2(s1);
 }
```




```
S::S(S &a)
{
    if(a.ptr==NULL) ptr=NULL;
    else {  ptr=new char [strlen(a.ptr)+1];
            strcpy(ptr,a.ptr);
         }
}
```

扫描全能王 创建

```
using namespace std;
#include <cstring>
#include <iostream>
class Person{
  char *name;
public:
 Person(char *s)
  {  name=new char [strlen(s)+1];
     strcpy(name,s); }

void GetName(char *s)
   { strcpy(s,name);}

 ~Person(){delete name;}

void Copy(Person s)
 { if(name!=NULL) delete name;
   name= new char[strlen(s.name)+1];
   strcpy(name,s.name); }
};
```

```
int main()
{
    Person x ("张三"),y ("李四");
     y.Copy(x);
 }
```

s前的&不能省，当采用类对象值传递时，会自动调用拷贝构造函数，函数结束前，自动调用析构函数



程序执行出错！

扫描全能王 创建

10.3.2 赋值操作函数

- 当同类对象之间执行赋值操作时，调用赋值操作函数。
- 赋值操作函数的一般格式：

```
<类名>&<类名>::operator=(const <类名>&<对象名>)
{
    …        //赋值操作函数体
    return *this;
}
```

- 若类中没有显式定义赋值操作函数，编译器自动生成一个公有的赋值操作函数，复制每个非静态数据成员。

扫描全能王 创建

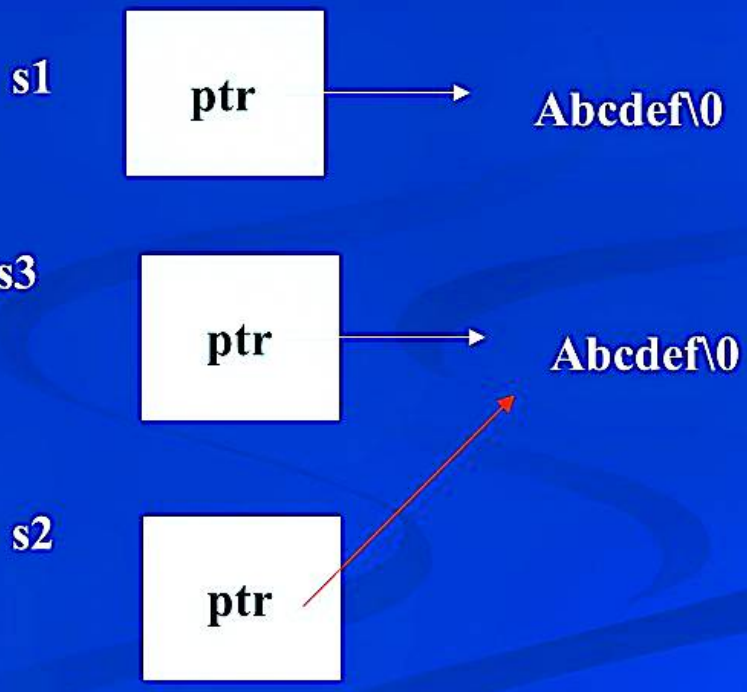
```
using namespace std;
#include <cstring>
#include <iostream>
class S{
   char *ptr;
public:
S(char *p=NULL)
       { if(!p) ptr=NULL;
         else {  ptr=new char[strlen(p)+1];
                 strcpy(ptr,p);}
       }
 ~S(){ delete ptr;}
S(S &a)
       { if(a.ptr==NULL) ptr=NULL;
         else {  ptr=new char [strlen(a.ptr)+1];
                 strcpy(ptr,a.ptr);}
       }
};
```

此处增加下面的成员函数：

```
S   &operator =(const S &a)
{ if(ptr) delet ptr;
  if(a.ptr==NULL) ptr=NULL;
  else {  ptr=new char [ strlen(a.ptr)+1];
          strcpy(ptr,a.ptr); }
  return *this;
}
```

```
int main()
{
 S s1("abcdef");
 S s3=s1,s2;
 s3.show();
 s2=s3;
 s2.show();
}
```

s2.oprator =(s3);



扫描全能王 创建

10.3.3 单参构造函数

```
using namespace std;
#include <iostream>
class Ex1{
    int x;
public:
    Ex1( int a)
      {x=a;
      cout<<“x=”<<x<<” 调用了构造函数!"<<endl;
      }
   ~Ex1()
   { cout<<"调用了析构函数!"<<endl;}
};

void main()
{ Ex1 x1(50);
  Ex1 x2=100;
   x2=200;
}
```

结果输出：

```
X=50 调用了构造函数!
X=100 调用了构造函数!
X=200 调用了构造函数!
调用了析构函数!
调用了析构函数!
调用了析构函数!
```

等同于x2(100)
的写法，条件是构造函数
的形参只有一个的情况下

x=200;的实际效果：(内部转换成如下的操作）

```
{ Ex1 tmp(200);
   x2=tmp;
}
```

当构造函数只有一个参数时，才能象以上这样使用。

扫描全能王 创建

构造函数有多个参数的情况：

```
using namespace std;
#include <iostream>
class Ex2{
   int x,y;
public:
   Ex2( int a,int b)
     {
          x=a;y=b;
          cout<<“x=”<<x<<“ y=”<<y<<“ 调用了构造函数!"<<endl;
     }
  ~Ex2()
     {
          cout<<"调用了析构函数!"<<endl;
     }
};
int main()
{
  Ex2 x1(50,100);
  x1= Ex2(300,600);
}
```

内部处理为：

```
{Ex2 tmp(300,600);
  x1=tmp;
}
```

输出结果：

```
x=50 y=100 调用了构造函数!
x=300 y=600 调用了构造函数!
调用了析构函数!
调用了析构函数!
```

扫描全能王 创建

多选题 2分

假设对象a 和b都是类A 的对象，并有函数A f();和 void f(A &x);。
下面哪些语句可能不执行类A 的拷贝构造函数（假设编译选项未启动返回值优化）？

A a=f();

B f(a);

C A c = a;

D A d(a);

E b = a;

扫描全能王 创建

10.4 复合函数与成员对象

- 在定义一个新类时，可把一个已定义类的对象作为该类的成员。产生这新定义类的对象时，也要对它的对象成员进行初始化，且只能通过这新类的构造函数来对它的所有数据成员初始化。要对对象成员进行初始化，必须通过调用其对象成员的构造函数来实现。

扫描全能王 创建

10.4复合函数与成员对象

- 新类构造函数形式为：

  类名：：类名（参数表）：c1(参数表)，…，cn(参数表)

  {

  函数体；

  }

- 注意：
  - c1 … cn的次序可任意，但习惯上按说明次序定义。
  - 当类自己不需要构造函数而其对象成员需构造初始化，则自己必须有一个构造函数，不能依赖于缺省的构造函数。

扫描全能王 创建

例：

```
using namespace std;
#include <iostream>
 class  A {
      int  x,y;
 public:
     A(int a,int b)  {x=a;y=b;}
};
class B{
     int Length,Width;
 public:
     B(int a,int b)
            {Length=a;Width=b;}
};
 class C{
     int r,High;
     A  a1;
     B  b1;
 public:
     C(int a,int b, int c, int d,int e,int f):a1(e,f),b1(c,d)
           { r=a;High=b;}
};
int  main(void)
{
  C c1(25,35,45,55,65,100);
}
```

❖当建立类C的对象时，先调用各个对象成员的构造函数(对对象成员的构造函数的调用顺序取决于这些对象成员在类中说明的顺序)，初始化相应的对象成员，然后才执行类C的构造函数，初始化类C中的其它成员；

❖析构函数的调用顺序与构造函数正好相反。

扫描全能王 创建

10.4复合函数与成员对象

- 三维坐标中的一条直线的可通过该直线的两个端点的坐标(x1,y1,z1)和(x2,y2,z2)来描述。定义一个类，实现坐标数据的初始化，输出端点的坐标和直线的长度。

扫描全能王 创建

```
using namespace std;
#include <iostream>
#include <math>
class Point{
    double x,y,z;
public:
    Point(double a=0,double b=0,double c=0)
        { x=a;   y=b;    z=c; }
    void Set(double a,double b,double c)
        {  Point(a,b,c);}
    void Print()
        { cout<<"x="<<x<<"y="<<y<<"z="<<z<<endl;};
    double Distance(Point &t)
        { double r;
          r=(x-t.x)*(x-t.x)+(y-t.y)*(y-t.y)+(z-t.z)*(z-t.z);
          return sqrt (r); }
};
class Line{
    Point x1,x2;
public:
    Line(double a1,double b1,double c1,double a2,double b2,double c2):x1(a1,b1,c1),x2(a2,b2,c2)
        {}
    double Lenth()
        {  return x1.Distance(x2);  }
    void PrintLine()  { cout<<"Line"<<endl;
                        x1.Print();
                        x2.Print();  }
};
```

```
int main()
{
  Line l1(1,2,3,4,5,6);
  Line l2(2,3,4,8,9,10);
  l1.PrintLine();
  l2.PrintLine();
  cout<<"Lenth1="<<l1.Lenth()<<endl;
  cout<<"Lenth2="<<l2.Lenth()<<endl;
}
```

```
Line:
x=1 y=2 z=3
x=4 y=5 z=6
Line:
x=2 y=3 z=4
x=8 y=9 z=10
Lenth1=5.19615
Lenth2=10.3923
```

扫描全能王 创建

10.5 对象数组

- 对象数组就是一类对象的一个有序集合，数组中所有元素是同一类的对象。
- 一维对象数组的定义：

  <类型><数组名>[<常量表达式>][={初始化列表}];

  例：

  Point points[3]={Point(1,2),Point(3,4)};

扫描全能王 创建

10.5 对象数组

- 对象数组可以作为一个类的数据成员。
- 注意：
  - 该类的构造函数中不能对各个数据元素进行显式初始化。
  - 数组元素的类应提供默认构造函数。

扫描全能王 创建

在Point 类的基础上，设计实现一个矩形类Rectangle，并测试该类。该类要求有下述成员：

- 数据成员：左上角点(x, y)，宽度width 和高度high，可用int 类型。
- 成员函数：对宽度和高度属性的读getXxx 和写setXxx，另外还有：
  - 构造函数。
  - void moveOff(int xoff , int yoff)：相对移动。
  - float getArea():计算矩形的面积。
  - float getPerimeter():计算矩形的周长。
  - void show(): 显示矩形的数据成员，以及面积和周长。

扫描全能王 创建

10.6 静态成员

类说明中可以用static 说明数据成员。不同的类对象对非静态成员存有不同的副本，而对静态成员只有一个副本。使用时并不需要指明哪个特定的对象。
甚至在没有类对象时也可使用静态成员。（此时用类名：：对象成员名）

静态的数据成员说明不是一个定义，所以要在其它地方对它定义。（分配存储区和提供初值，在类外说明），可以理解为静态数据是该类对象（变量）共有的。

主要用途：
1）记录类的所有对象所公用的数据
2）减少可访问的全局变量的个数

扫描全能王 创建

```
using namespace std;
#include <iostream>
class  A{
    int  i,j;
    static int x,y;
public:
   A(int a=0,int b=0,int c=0, int d=0)
   { i=a;j=b;x=c;y=d;}
  void Show()
{   cout << "i="<<i<<'\t'<<"j="<<j<<'\t ';
    cout << "x="<<x<<'\t'<<"y="<<y<<endl;
}
};
int A::x=0;
int A::y=0;
int  main( )
{
 A  a(2,3,4,5);
 a.Show();
 A  b(100,200,300,400);
 b.Show();
 a.Show();
}
```

对静态成员数据x、y的定义性说明

输出：

```
i=2       j=3       x=4       y=5
i=100     j=100     x=300     y=400
i=2       j=3       x=300     y=400
```

- 类的静态成员数据是静态分配存储空间的；
- 必须在文件作用域中，对静态成员数据作一次且只能作一次定义性说明；
- 在给静态成员数据作定义性说明时可以指定一个初值（缺省初值为0），这不受该成员数据访问权限的限制。

扫描全能王 创建

```
using namespace std;
#include <iostream>
class  A{
    int  i,j;
public:
 static int x;
   A(int a=0,int b=0)
   { i=a;j=b;}
   void Show()
    {  cout << "i="<<i<<'\t'<<"j="<<j<<endl;
       cout << "x="<<x<<endl;
    }
};

int A::x;

int  main( )
{
  cout <<"A::x="<<A::x<<endl;
}
```

输出：
A::x=0

- 因为静态成员数据在定义性说明时已分配了存储空间，所以通过静态成员数据名前加上类名和作用域运算符（::），可直接使用静态成员数据；
- 静态成员数据在除定义性说明时，受访问权限的约束。

扫描全能王 创建

利用静态成员数据作为产生对象的计数器：

```
using namespace std;
#include <iostream>
class  A{
   int  i;
   static count;
 public:
  A(int a=0)
    {     i=a;
          count++;
          cout <<"Number of Objects="<<count<<endl; }
  ~A(){ count --;
          cout <<"Number of  Objects="<<count<<endl; }
   void SetData(int a ){i=a;}
   void Show()
   {    cout << "i="<<i<<'\n';
        cout << "count="<<count<<endl;}
};

int A::count;
```

```
int  main( )
{
    A  a1(100),b[2];
}
```

执行程序时的输出为：

```
Number of Objects=1
Number of Objects=2
Number of Objects=3
Number of Objects=2
Number of Objects=1
Number of Objects=0
```

扫描全能王 创建

单选题 1分

下面代码输出结果为：

```
#include <iostream>
using namespace std;
class MyClass {
    int idno = nextid++;
    static int nextid;
public:
    MyClass() {}
    MyClass(const MyClass& obj){}
    MyClass(int) {}
    void print() { cout<<idno<<endl; }
};
int MyClass::nextid = 1;
int main() {
    MyClass a1, a2;
    MyClass a3{a2}; a3.print();
    MyClass a4(3); a4.print();
    return 0;
}
```

A 2 2

B 2 3

C 3 4

D 4 4

扫描全能王 创建

10.7.1 修饰符const

- const成员函数
  - <返回类型><函数名>(<形参表>)const{...}
  - const成员函数体中不能改变当前对象的数据成员，只能调用const成员函数。
- const对象
  - const <类名> 对象名或对象指针或对象引用
  - 常量对象
- const形参
  - 函数体中不会改变const形参的数据成员的值。
- const返回值
  - 函数的返回值不可改变，即调用方不能通过返回的指针或引用来改变返回对象的状态。

扫描全能王 创建

10.8 指针成员

- 在一个类中，指针可作为该类的数据成员，可以是单个指针，也可是指针数组。
- 若一个类中有指针成员，通常默认拷贝构造函数、赋值操作函数、析构函数需要显式定义，以避免运行错误。

扫描全能王 创建